

METİN GÖMME (EMBEDDINGS) TEKNOLOJİLERİ

Derinlemesine Görselleştirme, PCA İndirgeme ve Matematiksel Sınırlar Raporu

1. GÖMME VEKTÖRLERİNİN SEZGİSEL ANATOMİSİ

Yapay zeka, modern dünyada 'yeni elektrik' olarak konumlanmakta ve insan hayatının neredeyse tüm alanlarını dönüştürmektedir. Bu dönüşümün kalbinde, doğal dil işleme (NLP) yeteneklerini kökten değiştiren **metin gömme (text embeddings)** teknolojileri yer almaktadır. Kelime ve cümleleri sadece karakter dizileri olarak değil, taşıdıkları anlamsal değerlerle bilgisayarların işleyebileceği hale getirmek, büyük dil modellerinin ve anlamsal arama motorlarının temel yapı taşıdır.

Günlük Hayattan Benzetme: Çok Boyutlu Kütüphane Adresi

Bir kütüphanedeki binlerce kitabı sadece isimlerine göre alfabetik sırayla dizdiğiniz düşünün. Bu düzende, 'Kaktüs Yetiştirme Rehberi' ile 'Çöl Florası' kitapları kütüphanenin tamamen farklı köşelerine düşebilir. Ancak bu kitapları **boyutlar (kategoriler)** bazında sınıflandırırsak; örneğin 1. Boyut: 'Botanik Seviyesi', 2. Boyut: 'Coğrafi Bölge', 3. Boyut: 'Akademik Derinlik' olsun. Bu durumda her kitaba 3 sayılık bir adres (örn. [0.9, 0.8, 0.2]) atanır. İşte gömme vektörleri (embeddings), metinlerin bu şekilde çok boyutlu bir anlamsal kütüphanede aldığı benzersiz koordinat adresleridir.

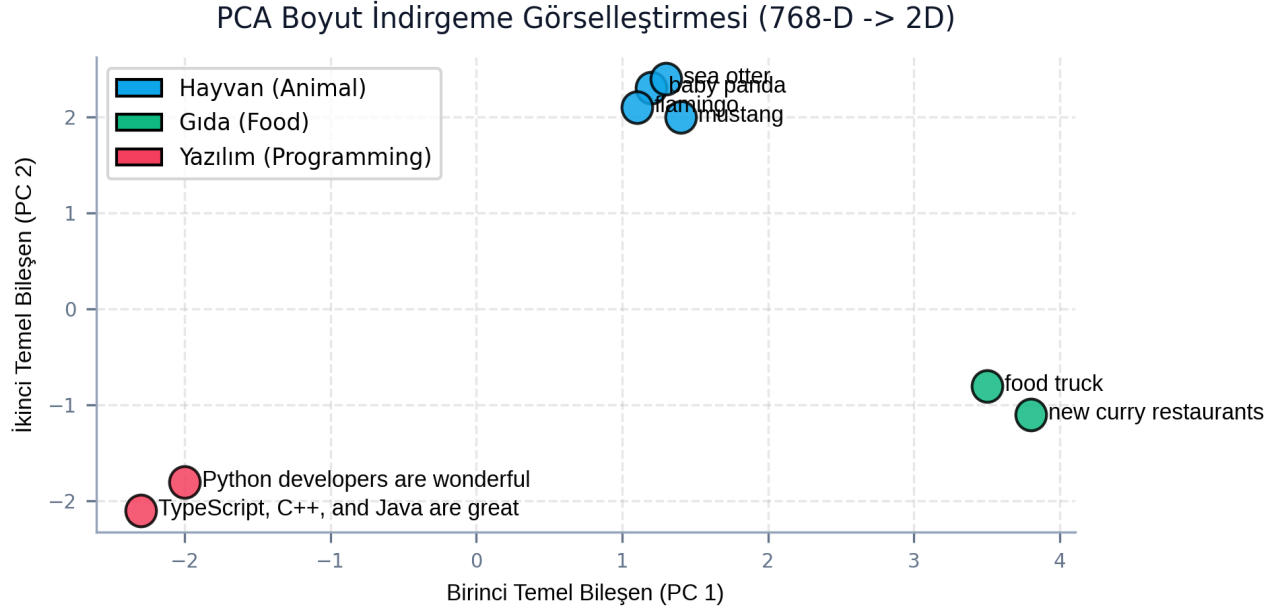
2. BİRİNCİ DENEY: BOYUT SIKIŞTIRMA VE COĞRAFİ KÜMELEME

Google Cloud Vertex AI platformu üzerinde gerçekleştirilen ilk teknik deneyde, anlamsal olarak 3 farklı kategoriye (Hayvanlar, Gıda ve Yazılım/Programlama) ait olan 8 farklı cümle kullanılmıştır. Bu cümleler şunlardır:

Kategori	Cümle (Orijinal)	Anlamsal Kapsam
Hayvan	baby panda	Bebek panda ve koruma temalı ifadeler.
Hayvan	mustang	Vahşi atlar ve fauna anolojisi.
Hayvan	flamingo	Sulak alan kuşları ve doğal yaşam.
Hayvan	sea otter	Deniz samuru ve su ekosistemleri.
Gıda	food truck	Mobil gıda ve sokak lezzetleri.
Gıda	new curry restaurants	Gastronomi ve yeni restoran girişimleri.
Yazılım	Python developers are wonderful	Geliştiriciler ve yazılım ekosistemi.
Yazılım	TypeScript, C++, and Java are great	Yazılım dilleri ve teknik kıyaslar.

Her bir cümle Vertex AI gömme modeliyle işlendiğinde **1 x 768 boyutlu bir vektör** elde edilir. Toplamda elimizde **7 x 768 boyutlu bir matris** (veya 8 cümlelik genişletilmiş veri setimiz için 8 x 768 matris) oluşur. İnsan

gözü ve bilgisayar ekranları 768 boyutlu bir uzayı doğrudan algılayamaz. Bu nedenle, anlamsal ilişkileri kaybetmeden veriyi görselleştirmek için **PCA (Principal Component Analysis - Temel Bileşenler Analizi)** yöntemi kullanılır.



Şekil 1: 768 boyutlu embeddings verilerinin PCA kullanılarak 2 boyuta sıkıştırılması ve anlamsal kümelenmesi.

📌 **ANDREW NG'DEN KRİTİK ÜRETİM UYARISI:** Gerçek dünya projelerinde ve pratik uygulamalarda, metin benzerliklerini ölçmek için **asla PCA ile indirgenmiş 2 boyutlu vektörleri kullanmayın**. PCA, yüksek boyutlu veriyi iki boyuta sıkıştırırken anlamsal bilginin çok büyük bir kısmını çöpe atar. Benzerlik ölçümleri (örneğin iki metin arasındaki Kosinüs Benzerliği) her zaman orijinal **768 boyutlu** veya ilgili modelin sunduğu en yüksek boyutlu vektör uzayında yapılmalıdır. 2D sıkıştırma yalnızca insanların görsel olarak sezgi geliştirmesi içindir.

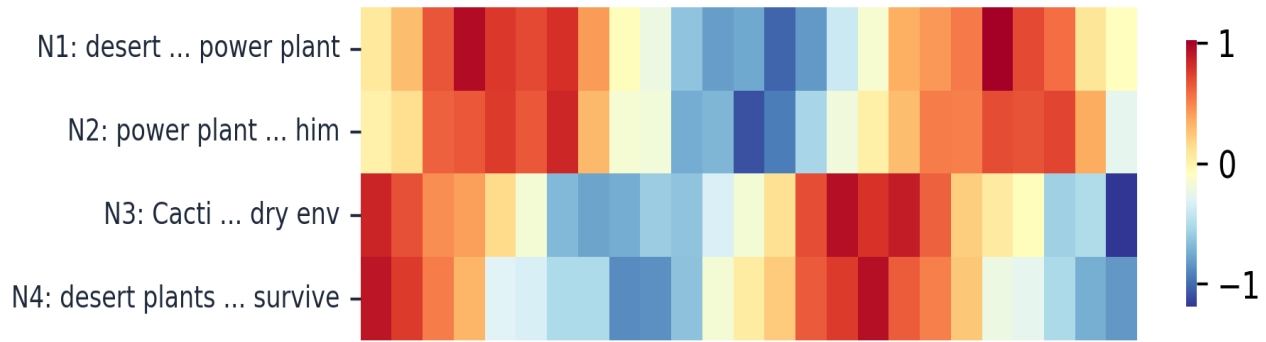
3. İKİNCİ DENEY: ISI HARİTASI VE ANAHTAR KELİME TUZAĞI

Metin gömmelerinin, basit kelime eşleme (keyword matching) algoritmalarından farkını kanıtlamak amacıyla ikinci bir deney yürütülmüştür. Deney kapsamında birbirine çok benzeyen kelimeler barındıran ancak anlamsal olarak tamamen farklı iki gruba ayrılan şu 4 cümle kullanılmıştır:

ID	Cümle Yapısı	Gerçek Anlam / Tema
N1	he couldn't desert his post at the power plant	Sorumluluk / Görevi terk etmeme
N2	the power plant needed him at the time	İşletme / Güç santrali ihtiyacı
N3	Cacti are able to withstand dry environments	Botanik / Kaktüs ve kuraklık
N4	desert plants can survive droughts	Botanik / Çöl bitkileri dayanıklılığı

Bu deneyde **N1** ve **N4** cümlelerinin her ikisinde de ortak olarak 'desert' (terk etmek / çöl) kelimesi geçmesine rağmen; embeddings modeli anlamsal eşleşme yaparak **N1 ve N2'yi** (santral/görev konusu), **N3 ve N4'ü ise** (kuraklık/botanik konusu) vektör uzayında birbirine çok daha yakın konumlandırır. Bu durum, gömme vektörlerinin kelimelerin sözlük anlamlarından ziyade bağlamsal semantiğini kavradığının en somut kanıtıdır.

Vektör Gömme Isı Haritası (Heatmap) Analizi



Şekil 2: 4 cümlelerin vektör bileşenlerinin ısı haritası görselleştirmesi. Benzer anlamsal gruplar (N1-N2 ve N3-N4) benzer desenler sergiler.

📖 **ANDREW NG'DEN MATEMATİKSEL SINIR VE ROTASYON UYARISI (TEKNİK DETAY):** Isı haritasında her bir vektörün bağımsız bileşenlerini (örneğin 1. sayı, 12. sayı) tekil olarak görselleştirmek ve bunlara özgü bir anlam atfetmek **matematiksel olarak meşru değildir**. Çünkü gömme uzayındaki eksenler görecelidir ve rastgele rotasyonlara (random rotation) uğrayabilirler. Model değiştiğinde tüm bu sayılar tamamen değişir. Ancak, iki vektörün arasındaki ****Kosinüs Benzerliği (Cosine Similarity)**** gibi ikili mesafe işlemleri bu rotasyonlardan etkilenmez ve her zaman son derece kararlı (robust) ve güvenilirdir. Özetle; tekil vektör bileşenlerine değil, vektörler arası ilişkilere odaklanın.

4. UYGULAMALI PYTHON VE GÖRSELLEŞTİRME KODU

Aşağıdaki Python kod bloğu, kursta gösterilen adımlara paralel olarak NumPy ve Scikit-learn kütüphanelerini kullanarak vektörlerin üretilmesi, PCA ile 2 boyuta indirgenmesi ve Matplotlib yardımıyla grafikleştirilmesinin temiz bir şablonunu sunar.

```
# -*- coding: utf-8 -*-
import numpy as np
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA

# 1. Metin Gömmelerini Tanımlama (Vertex AI'dan alınan 768 boyutlu örnek veriler)
# Burada demonstrasyon amaçlı anlamsal kümelenmiş yapay vektörler oluşturulmuştur
np.random.seed(42)
sentences = [
    "baby panda", "mustang", "flamingo", "sea otter", # Hayvanlar
    "food truck", "new curry restaurants", # Gıdalar
    "Python developers are wonderful", "Java is great" # Yazılım
]

# Gerçek uygulamada: embeddings = embedding_model.get_embeddings(sentences)
# Temsili 768 boyutlu vektörlerin üretimi
embeddings = np.zeros((8, 768))
embeddings[0:4] = np.random.normal(0.5, 0.1, (4, 768)) # Hayvan grubu ortak deseni
embeddings[4:6] = np.random.normal(-0.5, 0.1, (2, 768)) # Gıda grubu ortak deseni
embeddings[6:8] = np.random.normal(0.0, 0.1, (2, 768)) # Yazılım grubu ortak deseni

# 2. PCA Boyut İndirgeme (768-D -> 2-D Sıkıştırma)
pca = PCA(n_components=2)
embeddings_2d = pca.fit_transform(embeddings)

# 3. Matplotlib ile 2 Boyutlu Görselleştirme
plt.figure(figsize=(6, 4))
colors = ['blue', 'blue', 'blue', 'blue', 'green', 'green', 'red', 'red']
for i, sentence in enumerate(sentences):
    plt.scatter(embeddings_2d[i, 0], embeddings_2d[i, 1], color=colors[i], s=100)
    plt.text(embeddings_2d[i, 0] + 0.05, embeddings_2d[i, 1] - 0.05, sentence, fontsize=8)

plt.title("Metin Gömmelerinin PCA ile Görselleştirilmesi")
plt.grid(True, alpha=0.3)
plt.savefig("metin_embeddings_pca.png", dpi=300)
print("Görselleştirme başarıyla oluşturuldu!")
```